



大数据存储技术课程报告

Raft KV

基于 Raft 协议的分布式 KV 数据库 设计、实现与可视化验证报告

课程名称： 大数据存储技术
作业题目： 基于 Raft 的分布式 KV 数据库实现
学生姓名：
学号：
完成时间： 2026 年 6 月

目录

1	引言	1
1.1	作业背景	1
1.2	设计目标	1
1.3	系统功能概述	2
2	系统架构设计	2
2.1	总体架构	2
2.2	模块划分与职责	3
2.3	节点通信结构	3
2.4	可视化控制台设计	4
2.5	核心数据结构	4
3	Raft 协议实现细节	5
3.1	形式化约束与关键公式	5
3.2	节点状态转换	6
3.3	Leader 选举流程	6
3.4	RequestVote 实现	7
3.5	AppendEntries 与心跳机制	8
3.6	日志复制机制	8
3.7	日志提交与状态机应用	8
3.8	持久化与节点恢复	9
4	KV 状态机设计	10
4.1	Put 操作流程	10
4.2	Get 操作流程	10
4.3	Delete 操作流程	11
4.4	Follower 请求转发与错误处理	11
5	关键问题与解决方案	11
5.1	网络分区处理	11
5.2	日志冲突解决	11
5.3	选票分裂预防	12
5.4	Leader 宕机与重新选举	12
5.5	节点重启后的日志恢复	12
5.6	脑裂问题避免	13

6 测试方案与结果	13
6.1 测试策略	13
6.2 运行过程与截图证明	13
6.2.1 环境与单元测试	13
6.2.2 三节点启动与初始选主	14
6.2.3 基础 KV 操作	16
6.2.4 日志复制	16
6.2.5 Follower 宕机	17
6.2.6 Leader 宕机与新 Leader 可用性	18
6.2.7 旧 Leader 恢复与脑裂避免	20
6.2.8 快照字段截图	21
6.2.9 Lease Read 截图	21
6.2.10 动态添加节点截图	22
6.2.11 自动化验收与画图结果	23
6.2.12 可视化控制台截图	23
6.3 自动化测试结果	24
6.4 故障恢复实验结果	25
6.5 快照、租约读与成员变更验证	25
7 局限性与未来方向	26
8 总结	26
9 参考文献	27

1 引言

1.1 作业背景

分布式存储系统通常需要在多个副本之间保持数据一致，并在单个节点故障时继续提供服务。对于 KV 数据库而言，如果不同副本对同一 key 的写入顺序不一致，就可能出现读取结果不确定、故障恢复后数据回退等问题。因此，课程作业要求实现一个基于 Raft 协议的轻量级分布式 KV 数据库，用于理解一致性协议在副本复制、故障恢复和高可用场景中的作用。

Raft 协议将一致性问题拆分为 Leader 选举、日志复制和安全性约束三个主要部分，其设计目标是提升共识算法的可理解性（参考文献 [1][2]）。相比 Paxos，Raft 的角色划分和执行流程更直观，适合作为教学项目实现。本项目围绕可配置多节点集群完成核心 Raft 流程，并通过 HTTP 接口提供 Put、Get、Delete 操作。

1.2 设计目标

本项目的设计目标如下：

- 支持至少 3 个节点组成集群，并可按配置扩展到 5 个或更多节点；
- 支持 Follower、Candidate、Leader 三种角色；
- 实现随机选举超时、Leader 选举和心跳维持；
- 实现 RequestVote 与 AppendEntries 两类 Raft RPC；
- 实现日志复制、日志匹配检查、冲突日志截断和多数派提交；
- 支持 Put、Get、Delete 三种 KV 操作；
- 写操作必须经过 Raft 日志提交后才能应用到状态机；
- 任意 1 个节点宕机后系统仍可继续服务；
- Leader 宕机后剩余节点可以重新选举；
- 节点状态、日志、提交进度、快照和成员列表持久化到本地 JSON 文件；
- 实现快照压缩、Lease Read 和教学版动态成员变更；
- 提供浏览器可视化控制台，用于观察集群状态、执行 KV 操作和演示成员变更。

从一致性角度看，本项目采用“Leader 串行化写入 + 多数派提交 + 状态机顺序应用”的设计。设当前成员数为 N ，多数派大小定义为

$$Q(N) = \left\lfloor \frac{N}{2} \right\rfloor + 1.$$

只有当同一条日志至少复制到 $Q(N)$ 个成员后，Leader 才允许推进提交位置。三节点基础集群中 $Q(3) = 2$ ，因此可以容忍 1 个节点故障；当配置扩展到五节点时， $Q(5) = 3$ ，系统可以在两个非 Leader 节点不可用时继续形成多数派。多数派计算由成员列表动态决定，而不是写死为固定三节点。

1.3 系统功能概述

系统使用 Go 语言实现，HTTP 服务与 JSON 编解码主要依赖 Go 标准库（参考文献 [3][4]）。节点通过命令行参数指定 ID，并读取集群配置文件获取节点列表。每个节点提供两个 HTTP 服务：客户端 API 服务和 Raft RPC 服务。客户端通过 API 端口访问状态查询、Leader 查询、KV 读写、成员管理和内置 Web 控制台；节点之间通过 RPC 端口访问投票、日志复制和快照安装接口。

写请求只能由 Leader 处理。Leader 将写操作封装为日志项，复制到 Follower，并在多数节点复制成功后推进 `commitIndex`，再应用到 KV 状态机。Follower 收到客户端写请求时返回当前 Leader 信息，由客户端重试。

2 系统架构设计

2.1 总体架构

系统默认采用三节点对等部署方式，同时提供五节点配置用于演示节点数量不固定。任一时刻最多有一个 Leader 对外处理写请求，其他节点作为 Follower 接收日志复制、心跳和快照。为了让实验过程不仅停留在终端命令层面，节点 API 服务还内置了 Web 控制台；控制台本身不改变 Raft 协议路径，而是把已有的状态查询、Leader 路由、KV 操作和成员管理接口组织成可观察、可演示的操作界面。

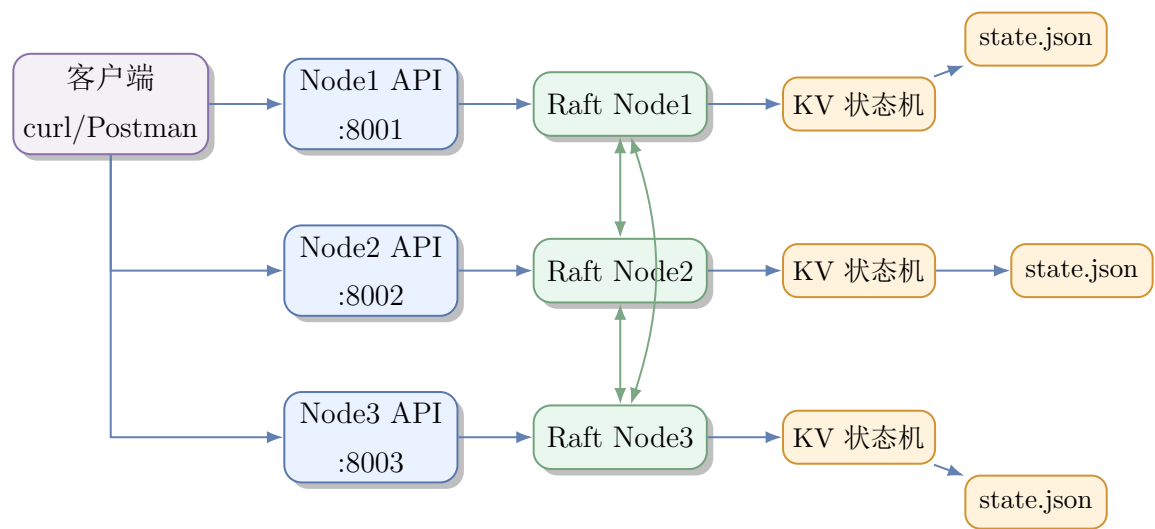


图 1: 系统总体架构图

2.2 模块划分与职责

表 1: 模块划分与职责

模块	主要职责
cmd/node	节点启动入口，解析命令行参数，读取集群配置，创建 Raft 节点并启动 HTTP 服务。
config	存放三节点和五节点集群配置，包括节点 ID、API 地址、Raft RPC 地址和持久化目录。
raft	实现 Raft 核心逻辑，包括角色状态、选举、心跳、日志复制、提交推进、持久化和 RPC 参数。
kv	实现 KV 状态机，支持 Put、Get、Delete。
server	提供客户端 HTTP API、节点间 Raft RPC 路由和内置 Web 控制台页面。
scripts	提供 Windows 和 Linux/macOS 的启动、停止和测试脚本。
tools	提供自动验收工具，启动临时真实集群，验证核心功能、快照、Lease Read、动态成员变更并生成可视化图表。
report	保存开发报告、运行截图和实验验证材料。

2.3 节点通信结构

每个节点同时监听两个端口。API 端口负责客户端请求，RPC 端口负责 Raft 节点间通信。将两类接口分开可以降低调试复杂度，也便于在测试中区分用户请求和协议内部请求。

客户端接口覆盖 Web 控制台、状态查询、KV 操作和成员管理：

```
GET /ui/  
GET /status  
GET /leader  
POST /kv/put  
GET /kv/get  
POST /kv/delete  
GET /cluster  
POST /cluster/add  
POST /cluster/remove
```

Listing 1: 客户端 API 与控制台入口

节点间接口负责 Raft 协议内部通信，包括 `POST /raft/request_vote`、`POST /raft/append_entries` 和 `POST /raft/install_snapshot`。

2.4 可视化控制台设计

内置控制台以单页静态 HTML 的形式嵌入节点二进制文件，访问任一节点的 `/ui/` 即可打开。页面默认以当前节点 API 地址作为观测入口，先调用 `/cluster` 获取成员列表，再分别访问各成员的 `/status`，把角色、任期、Leader、提交位置、日志长度和快照下标汇总为表格。这样，读者无需在多个终端中反复执行状态查询，也能直接观察三节点或动态扩容后的集群状态。

控制台中的写入、删除和成员变更操作仍遵循 Raft 约束：前端先查询 `/leader`，再把写请求发送到当前 Leader；如果连接节点返回 `not leader`，页面会根据响应中的 `leader_addr` 重试。该路径可以概括为

client command → Leader → Raft log → majority commit → state machine.

因此，可视化层只是改善观察和操作方式，并不绕过日志复制、多数派提交或状态机应用。由于浏览器页面可能从 8001 端口访问 8002 或 8003 的 Leader API，服务端统一补充了 CORS 和 OPTIONS 预检处理，保证前端可以在本机多端口集群中稳定演示。

2.5 核心数据结构

Raft 节点主要维护如下状态：

```

type Role string

const (
    Follower Role = "Follower"
    Candidate Role = "Candidate"
    Leader Role = "Leader"
)

type LogEntry struct {
    Index int `json:"index"`
    Term int `json:"term"`
    Command kv.Command `json:"command"`
}

```

Listing 2: Raft 角色与日志项

KV 命令结构如下：

```

type Command struct {
    Op string `json:"op"`
    Key string `json:"key"`
    Value string `json:"value,omitempty"`
}

```

Listing 3: KV 命令结构

节点状态包括 `currentTerm`、`votedFor`、日志、`commitIndex`、`lastApplied`、`nextIndex`、`matchIndex`、Leader ID、Peer 列表和 KV 状态机。共享状态使用互斥锁保护，网络请求使用带超时的 HTTP Client，避免节点故障时永久阻塞。

3 Raft 协议实现细节

3.1 形式化约束与关键公式

本项目遵循 Raft 的几个核心安全约束，并在实现中用任期、投票记录、日志匹配和多数派提交来保证：

- 选主条件：Candidate 在任期 t 中获得不少于 $Q(N)$ 票时才能成为 Leader，即

$$\text{votes}(\text{candidate}, t) \geq Q(N).$$

- **提交条件：**Leader 对当前任期日志 i 的提交条件为

$$\text{term}(i) = \text{currentTerm} \quad \wedge \quad |\{p \mid \text{matchIndex}_p \geq i\}| \geq Q(N).$$

该条件避免旧任期日志在新 Leader 上被错误地直接提交。

- **顺序应用：**状态机只应用满足

$$\text{lastApplied} < i \leq \text{commitIndex}$$

的日志，且按照日志下标递增顺序应用，保证每个副本最终执行相同命令序列。

- **故障容忍：**在只考虑崩溃故障时，成员数为 N 的集群最多可容忍

$$f_{\max} = \left\lfloor \frac{N-1}{2} \right\rfloor$$

个节点不可用；超过该数量时无法形成多数派，写入应超时或失败。

3.2 节点状态转换

节点初始角色为 Follower。Follower 如果在随机选举超时时间内没有收到合法 Leader 心跳，则转为 Candidate，增加当前任期并发起投票。Candidate 获得多数派投票后成为 Leader。Leader 周期性发送心跳维持权威。Candidate 或 Leader 收到更高任期的 RequestVote 或 AppendEntries 后必须退回 Follower。

3.3 Leader 选举流程

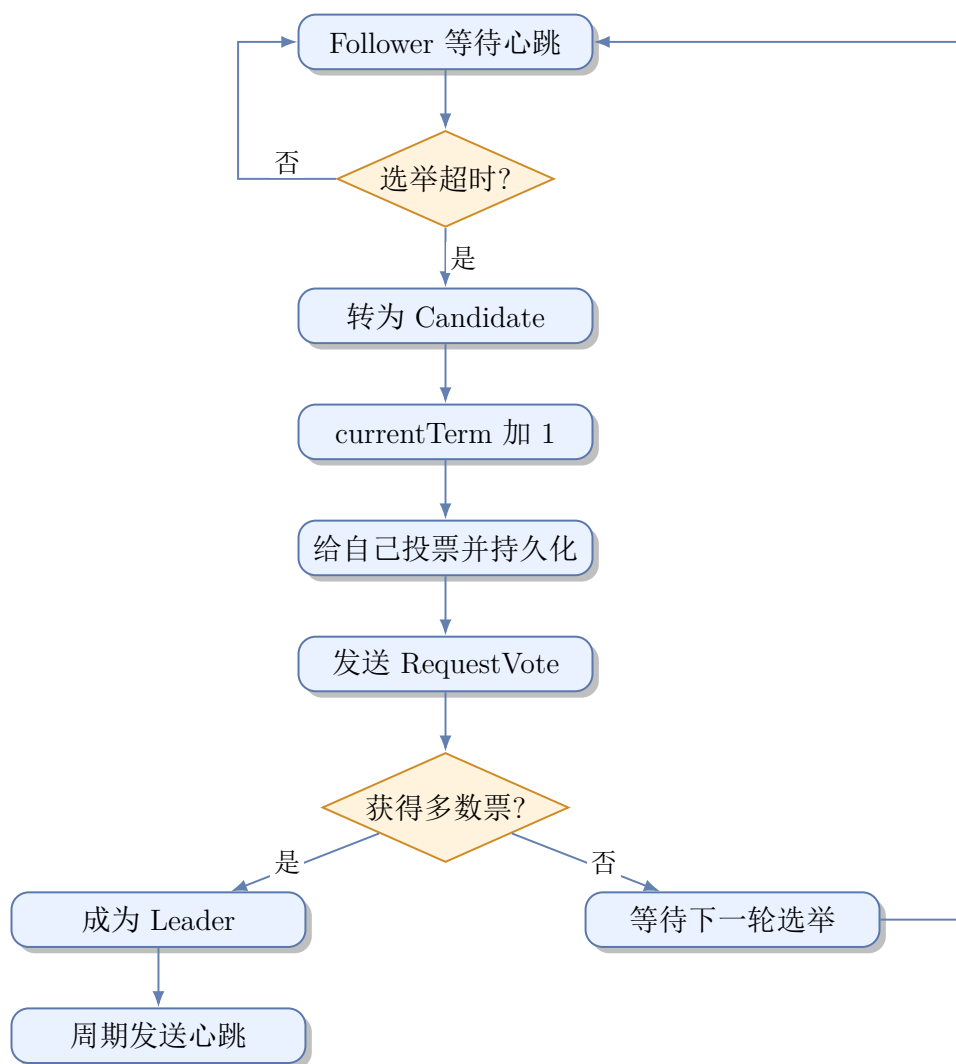


图 2: Leader 选举流程图

本项目的选举超时时间为 300ms 到 600ms 的随机值，心跳间隔为 100ms。随机超时降低了多个节点同时发起选举造成选票分裂的概率。

3.4 RequestVote 实现

RequestVote 请求包含任期、Candidate ID、最后日志下标和最后日志任期。节点收到投票请求时按如下规则判断：

- 请求任期小于当前任期，则拒绝；
- 请求任期大于当前任期，则更新本地任期并退回 Follower；
- 当前任期未投票，或已经投给同一 Candidate，才继续判断；
- Candidate 日志至少与本节点一样新时才投票；
- 投票后持久化 currentTerm 与 votedFor，并重置选举计时器。

3.5 AppendEntries 与心跳机制

Leader 使用 AppendEntries RPC 同时承担日志复制和心跳功能。没有新日志时，Leader 发送空 entries 作为心跳。Follower 收到合法 AppendEntries 后记录 Leader ID，并重置选举计时器。

AppendEntries 请求包含 PrevLogIndex、PrevLogTerm、日志 entries 和 LeaderCommit。Follower 先检查前置日志是否匹配，不匹配则拒绝；匹配后追加新日志，并根据 Leader 的提交位置推进本地 commitIndex。

3.6 日志复制机制

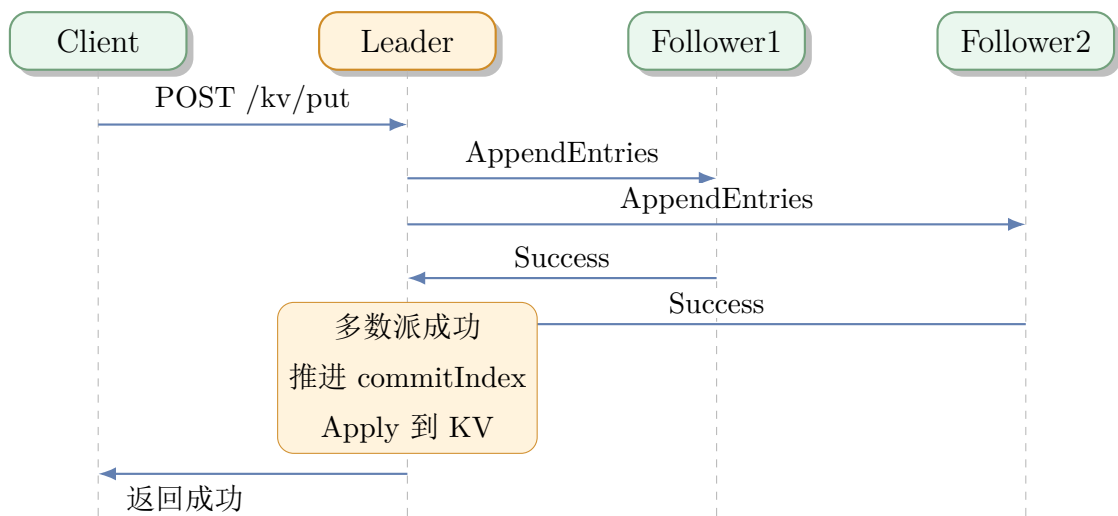


图 3: 日志复制时序图

Leader 为每个 Follower 维护 nextIndex 和 matchIndex。复制失败时降低 nextIndex 并重试；复制成功时更新该节点的 matchIndex。当某条当前任期日志被多数节点复制后，Leader 推进 commitIndex 并应用到状态机。

3.7 日志提交与状态机应用

日志只有在多数派复制成功后才被提交。节点调用 applyCommittedLocked 将所有已提交但未应用的日志依次应用到 KV 状态机。Put 操作写入 key/value，Delete 操作删除 key。这样可以避免未提交日志直接影响对外可见状态。对客户端而言，写请求返回成功意味着该命令已经进入已提交前缀；后续 Leader 读取本地状态机时不会看到未提交写入。

成员变更也作为特殊日志命令提交。AddNode 或 RemoveNode 日志被多数派提交后，节点才更新本地成员列表和多数派计算，避免单节点本地修改配置导致集群视图不一致。

以添加 node4 为例，成员变更先作为普通 Raft 日志进入已提交前缀，然后节点更新 `peers`，多数派大小由

$$Q(3) = 2$$

变为

$$Q(4) = 3.$$

新节点启动后通过 `AppendEntries` 或 `InstallSnapshot` 追赶 Leader 的提交位置。本项目的动态成员变更定位为教学版实现，用于展示“成员变更必须经由 Raft 日志提交后生效”这一核心思想；生产级并发扩缩容和 `joint consensus` 不纳入本次实现范围。

3.8 持久化与节点恢复

每个节点的持久化文件为 `data/nodeX/state.json`，保存字段包括：

- `currentTerm`: 当前任期；
- `votedFor`: 当前任期投票对象；
- `log`: 快照边界之后的日志；
- `commitIndex`: 已提交日志位置；
- `lastApplied`: 已应用日志位置；
- `snapshotIndex`、`snapshotTerm`、`snapshot`: 快照边界和状态机快照；
- `peers`: 当前成员列表。

节点重启时加载该文件，先恢复快照中的 KV 状态，再重放快照之后的已提交日志。日志达到阈值后生成状态机快照并截断旧日志，落后节点如果无法通过 `AppendEntries` 追赶，会通过 `InstallSnapshot` 恢复到快照边界。设快照阈值为 T ，当

$$\text{commitIndex} - \text{snapshotIndex} \geq T$$

时，节点将状态机序列化为快照，并令

$$\text{snapshotIndex}' = \text{commitIndex}, \quad \text{snapshotTerm}' = \text{term}(\text{commitIndex}).$$

压缩后只保留快照边界之后的日志，从而限制日志文件持续增长。

从空间复杂度看，如果不压缩，日志长度随写入次数 W 线性增长；引入快照阈值 T 后，本项目在稳定提交场景下只保留快照边界之后的日志，实际持久化日志长度近似满足

$$|\log| \approx \text{lastLogIndex} - \text{snapshotIndex} + 1.$$

因此，快照压缩不是未来改进项，而是当前系统已经实现的日志压缩与恢复机制。

4 KV 状态机设计

4.1 Put 操作流程

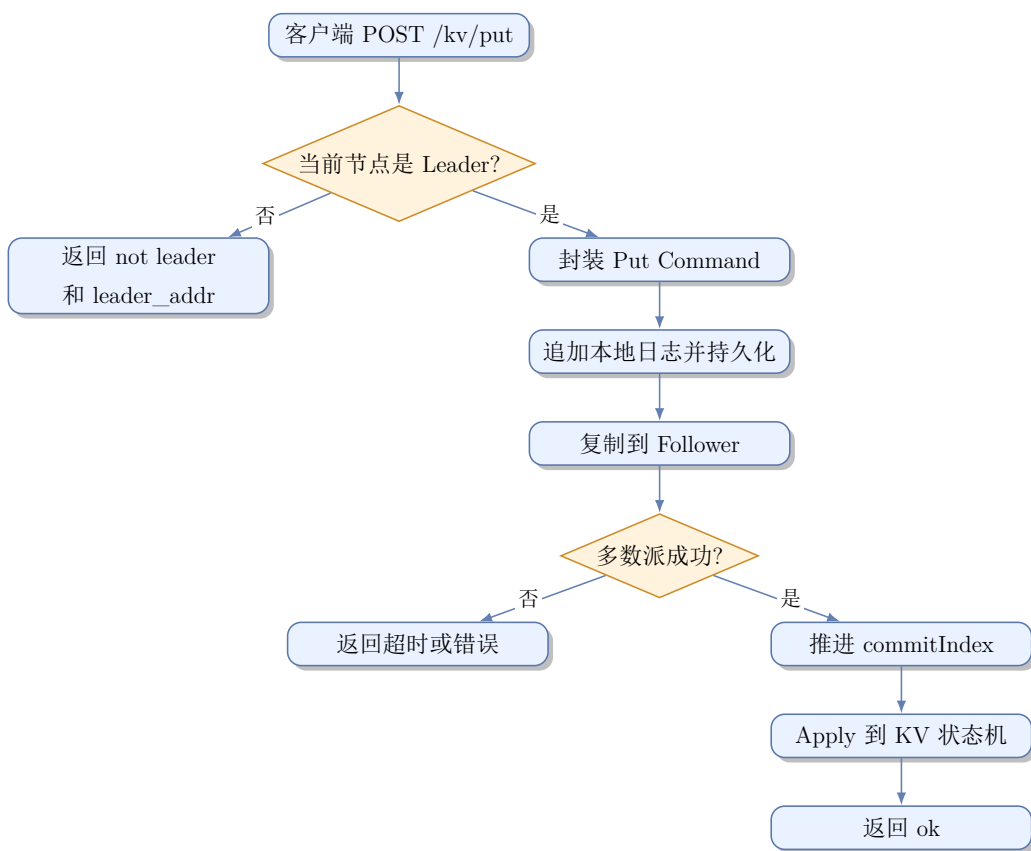


图 4: Put 请求处理流程图

4.2 Get 操作流程

Get 采用 Lease Read。Leader 在读取本地 KV 状态机前，通过心跳或 AppendEntries 确认自己仍能联系多数派；租约有效时不为读请求追加日志。Follower 收到 Get 请求时返回 Leader ID 和 Leader API 地址。

设最近一次多数派确认时间为 t_{ack} ，租约长度为 L ，当前时间为 now 。当

$$now < t_{\text{ack}} + L$$

时，Leader 认为租约仍有效，可以直接从本地状态机读取；否则会先向多数派复制空日志或心跳以刷新租约。本项目返回字段 `read_mode=lease_read`，用于在截图和测试中证明读路径没有追加普通写日志。

4.3 Delete 操作流程

Delete 与 Put 类似，被封装为日志命令并通过 Raft 复制。日志被多数派提交后，状态机删除对应 key。删除不存在的 key 不报错，视为幂等操作。

4.4 Follower 请求转发与错误处理

非 Leader 收到写请求时返回 HTTP 409，响应中包含 Leader 信息。例如：

```
{
  "error": "not leader",
  "leader_id": 1,
  "leader_addr": "127.0.0.1:8001"
}
```

Listing 4: Follower 返回 Leader 信息

5 关键问题与解决方案

5.1 网络分区处理

Raft 通过多数派机制避免少数派提交日志。即使旧 Leader 位于少数派，它也无法将新日志复制到多数节点，因此不会推进提交位置。多数派分区可以重新选举 Leader 并继续服务。

5.2 日志冲突解决

日志冲突通过 `PrevLogIndex` 与 `PrevLogTerm` 检查解决。Follower 如果发现前置日志不存在或任期不一致，会拒绝 `AppendEntries`。Leader 收到失败后降低该 Follower 的 `nextIndex` 并重试。当前置日志匹配后，Follower 截断冲突位置及之后日志，再追加 Leader 日志。

5.3 选票分裂预防

项目使用 300ms 到 600ms 的随机选举超时，降低多个 Follower 同时成为 Candidate 的概率。即使发生选票分裂，下一轮随机超时仍会重新拉开发起选举的时间。

5.4 Leader 宕机与重新选举

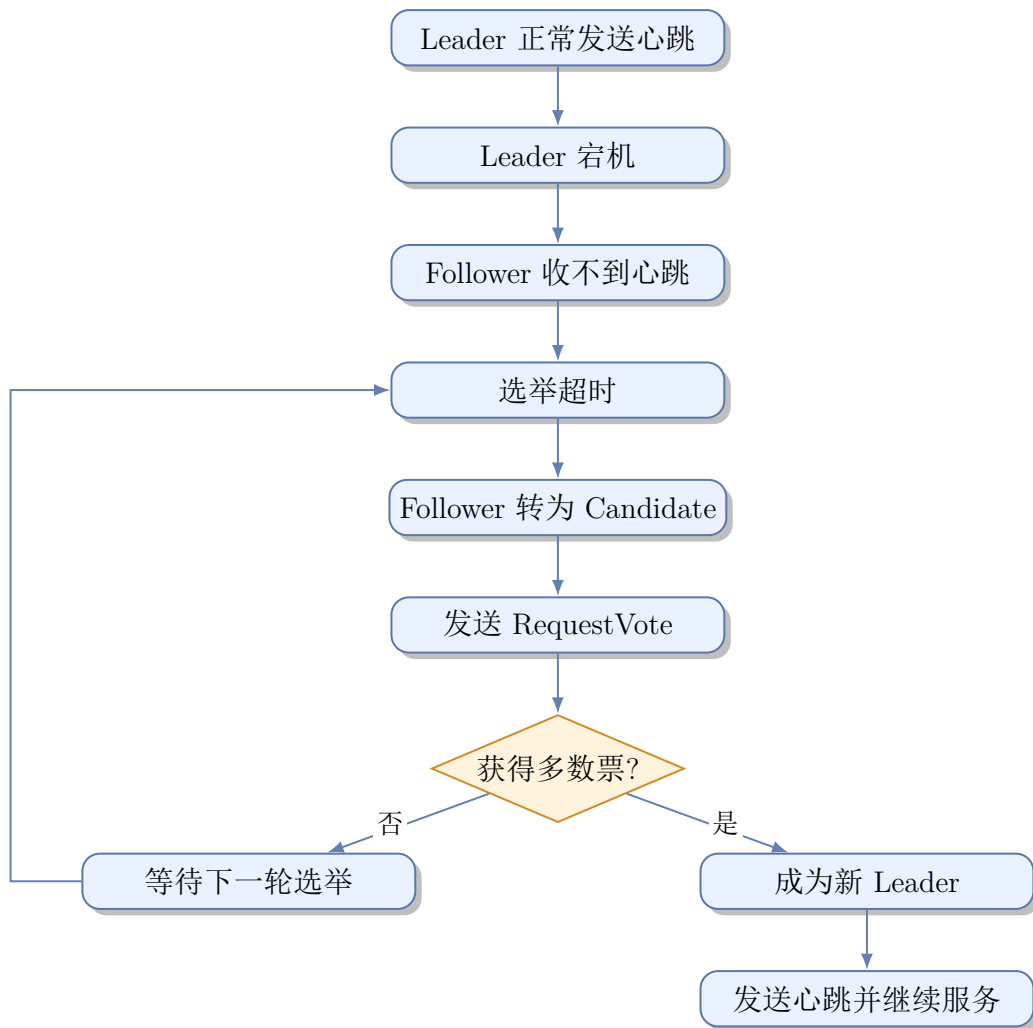


图 5: Leader 宕机恢复流程图

5.5 节点重启后的日志恢复

节点重启时加载本地 `state.json`，恢复任期、投票状态、成员列表、快照、日志、提交位置和已应用位置，并采用“恢复快照 + 重放快照后日志”的方式恢复状态机。恢复后的节点如果收到更高任期的 `AppendEntries`，会退回 Follower 并继续追赶 Leader 日志；如果本地日志已经落后到 Leader 的快照边界之前，则通过 `InstallSnapshot` 追赶。

5.6 脑裂问题避免

脑裂避免依赖三项机制：任期单调递增、同一任期单节点最多投一票、Leader 必须获得多数派。旧 Leader 恢复后，如果集群已经产生更高任期的新 Leader，则旧 Leader 会在收到更高任期 RPC 后退回 Follower。

6 测试方案与结果

6.1 测试策略

本项目的验证思路采用由内到外的分层方式。状态机层先验证 Put、Get、Delete 的语义以及从快照恢复后的数据一致性；Raft 层重点验证选举投票、日志匹配、冲突截断、多数派提交、快照安装和成员变更日志的应用；服务层则启动真实 HTTP/RPC 节点，让客户端请求、节点间 RPC、持久化文件和故障恢复共同参与测试。这样可以避免只在单个函数层面“看起来正确”，却在真实多节点通信中暴露端口、重试、重定向或持久化状态不一致的问题。

自动化测试之外，实验记录还包含真实截图证据。截图覆盖三节点启动、Leader 查询、KV 写读删、日志复制、Follower 宕机继续服务、Leader 宕机重选主以及旧 Leader 恢复后的单 Leader 状态；快照、Lease Read 与动态成员变更截图展示日志压缩、租约读、成员添加以及 node4 追赶状态；可视化截图展示浏览器控制台如何集中呈现集群状态，并将操作路由到 Leader。五节点场景用于验证多数派计算和配置读取不依赖固定三节点。

6.2 运行过程与截图证明

本节给出系统运行过程和截图证据。实验开始时启动 node1、node2、node3，集群选出 node2 作为 Leader；随后验证基本 KV 操作和日志复制；在停止一个 Follower 后，剩余两个节点仍能形成 $Q(3) = 2$ 的多数派并继续提交写入；当 Leader node2 停止后，node3 被选为新 Leader，并能继续对外处理 Put/Get；最后重启旧 Leader，node2 退回 Follower，系统仍保持唯一 Leader。快照字段、Lease Read、动态添加成员、node4 追赶状态以及 Web 控制台截图共同说明系统既能完成协议层复制与恢复，也能通过浏览器界面进行可视化观察和操作。

6.2.1 环境与单元测试

基础版本在 Windows 环境下使用 go1.26.4 windows/amd64，并通过 `go test ./...`。当前工程保留该测试入口，并额外补充了 `server` 包的真实 HTTP/RPC 多节点集成测试。


```
C:\Users\PC>go version
go version go1.26.4 windows/amd64
```

图 6: Go 版本截图: 使用 go1.26.4 windows/amd64

```
E:\Tongji learning materials\大三下\大数据存储技术\大作业\raft_kv_codex_task_package>D:\Go\bin\go.exe test ./...
?      raft-kv/cmd/node      [no test files]
ok      raft-kv/kv            1.310s
ok      raft-kv/raft          1.849s
?      raft-kv/server        [no test files]
```

图 7: 基础版本测试截图: go test ./... 通过

6.2.2 三节点启动与初始选主

启动 node1、node2、node3 后, 状态查询显示 node2 为 Leader, node1 与 node3 为 Follower, 三个节点返回的 leader_id 均为 2。

```
node1 role=Follower term=77 leader=2 leader_addr=127.0.0.1:8002
node2 role=Leader term=77 leader=2 leader_addr=127.0.0.1:8002
node3 role=Follower term=77 leader=2 leader_addr=127.0.0.1:8002
```

Listing 5: node2 初始 Leader 状态摘要

```
E:\Tongji learning materials\大三下\大数据存储技术\大作业\raft_kv_codex_task_package>scripts\start_cluster.bat
Starting Raft KV cluster...
Go command: D:\Go\bin\go.exe
Done. Wait 2 seconds, then visit:
http://127.0.0.1:8001/status
http://127.0.0.1:8002/status
http://127.0.0.1:8003/status
E:\Tongji learning materials\大三下\大数据存储技术\大作业\raft_kv_codex_task_package>

C:\raft-kv-node1 - "D:\Go\bin\go.exe" run ./cmd/node --id=1 --config=config/cluster.json
2026/06/17 21:29:55 [node 1] start role=Follower term=76 api=127.0.0.1:8001 raft=127.0.0.1:9001
2026/06/17 21:29:55 [node 1] api server listen on 127.0.0.1:8001
2026/06/17 21:29:55 [node 1] raft rpc server listen on 127.0.0.1:9001

C:\raft-kv-node2 - "D:\Go\bin\go.exe" run ./cmd/node --id=2 --config=config/cluster.json
2026/06/17 21:29:55 [node 2] start role=Follower term=76 api=127.0.0.1:8002 raft=127.0.0.1:9002
2026/06/17 21:29:55 [node 2] api server listen on 127.0.0.1:8002
2026/06/17 21:29:55 [node 2] raft rpc server listen on 127.0.0.1:9002
2026/06/17 21:29:55 [node 2] start election term=77
2026/06/17 21:29:55 [node 2] become Leader term=77

C:\raft-kv-node3 - "D:\Go\bin\go.exe" run ./cmd/node --id=3 --config=config/cluster.json
2026/06/17 21:29:55 [node 3] start role=Follower term=76 api=127.0.0.1:8003 raft=127.0.0.1:9003
2026/06/17 21:29:55 [node 3] api server listen on 127.0.0.1:8003
2026/06/17 21:29:55 [node 3] raft rpc server listen on 127.0.0.1:9003
2026/06/17 21:29:55 [node 3] vote granted to node 2 term=77
```

图 8: 三节点启动截图: node2 日志显示成为 Leader

```

C:\Users\PC>curl.exe http://127.0.0.1:8001/status
{"id":1,"role":"Follower","term":77,"leader_id":2,"leader_addr":"127.0.0.1:8002","commit_index":10,"last_applied":10,"log_len":11,"last_log_term":13}

C:\Users\PC>curl.exe http://127.0.0.1:8002/status
{"id":2,"role":"Leader","term":77,"leader_id":2,"leader_addr":"127.0.0.1:8002","commit_index":10,"last_applied":10,"log_len":10,"last_log_term":11}

C:\Users\PC>curl.exe http://127.0.0.1:8003/status
{"id":3,"role":"Follower","term":77,"leader_id":2,"leader_addr":"127.0.0.1:8002","commit_index":10,"last_applied":10,"log_len":10,"last_log_term":11}

```

图 9: CMD 状态查询截图: node2 为 Leader, node1 和 node3 为 Follower

```

C:\Users\PC>curl.exe http://127.0.0.1:8001/leader
{"leader_id":2,"leader_addr":"127.0.0.1:8002","is_leader":false}

C:\Users\PC>curl.exe http://127.0.0.1:8002/leader
{"leader_id":2,"leader_addr":"127.0.0.1:8002","is_leader":true}

C:\Users\PC>curl.exe http://127.0.0.1:8003/leader
{"leader_id":2,"leader_addr":"127.0.0.1:8002","is_leader":false}

```

图 10: CMD Leader 查询截图: 三个节点均返回 leader_id=2

```

(base) PS C:\Windows\System32\WindowsPowerShell\v1.0> $leader = @(foreach ($p in 8001,8002,8003) {
>> try {
>> $s = Invoke-RestMethod -Uri "http://127.0.0.1:$p/status" -TimeoutSec 1
>> if ($s.role -eq "Leader") { [pscustomobject]@{ Port = $p; Id = $s.id; Addr = $s.leader_addr } }
>> } catch {}
>> }) | Select-Object -First 1
(base) PS C:\Windows\System32\WindowsPowerShell\v1.0> $leaderPort = $leader.Port
(base) PS C:\Windows\System32\WindowsPowerShell\v1.0> $leader

Port Id Addr
---- --
8002 2 127.0.0.1:8002

```

图 11: PowerShell Leader 自动探测截图: 脚本识别 node2 的 API 端口为 8002

```

(base) PS C:\Windows\System32\WindowsPowerShell\v1.0> Invoke-RestMethod http://127.0.0.1:8001/status | ConvertTo-Json -C
ompress
{"id":1,"role":"Follower","term":77,"leader_id":2,"leader_addr":"127.0.0.1:8002","commit_index":10,"last_applied":10,"lo
g_len":11,"last_log_term":13}
(base) PS C:\Windows\System32\WindowsPowerShell\v1.0> Invoke-RestMethod http://127.0.0.1:8002/status | ConvertTo-Json -C
ompress
{"id":2,"role":"Leader","term":77,"leader_id":2,"leader_addr":"127.0.0.1:8002","commit_index":10,"last_applied":10,"log_
len":10,"last_log_term":11}
(base) PS C:\Windows\System32\WindowsPowerShell\v1.0> Invoke-RestMethod http://127.0.0.1:8003/status | ConvertTo-Json -C
ompress
{"id":3,"role":"Follower","term":77,"leader_id":2,"leader_addr":"127.0.0.1:8002","commit_index":10,"last_applied":10,"lo
g_len":10,"last_log_term":11}

```

图 12: PowerShell 状态查询截图: 三节点角色与提交进度保持一致

```
(base) PS C:\Windows\System32\WindowsPowerShell\v1.0> Invoke-RestMethod http://127.0.0.1:8001/leader | ConvertTo-Json -Compress
{"leader_id":2,"leader_addr":"127.0.0.1:8002","is_leader":false}
(base) PS C:\Windows\System32\WindowsPowerShell\v1.0> Invoke-RestMethod http://127.0.0.1:8002/leader | ConvertTo-Json -Compress
{"leader_id":2,"leader_addr":"127.0.0.1:8002","is_leader":true}
(base) PS C:\Windows\System32\WindowsPowerShell\v1.0> Invoke-RestMethod http://127.0.0.1:8003/leader | ConvertTo-Json -Compress
{"leader_id":2,"leader_addr":"127.0.0.1:8002","is_leader":false}
```

图 13: PowerShell Leader 查询截图：三个节点均确认 node2 为 Leader

6.2.3 基础 KV 操作

向 Leader 写入 name=raft 后，Get 可以读到该值；Delete 后再次 Get 返回 404。该场景同时保留 PowerShell 和 CMD 两种终端截图，便于复现实验命令。

```
(base) PS C:\Windows\System32\WindowsPowerShell\v1.0> Invoke-RestMethod -Method Post -Uri "http://127.0.0.1:$leaderPort/kv/put" -ContentType "application/json" -Body '{"key":"name","value":"raft"}'

index  ok term
-----
11 True  77

(base) PS C:\Windows\System32\WindowsPowerShell\v1.0> Invoke-RestMethod -Uri "http://127.0.0.1:$leaderPort/kv/get?key=name"

found key  value
-----
True name raft

(base) PS C:\Windows\System32\WindowsPowerShell\v1.0> Invoke-RestMethod -Method Post -Uri "http://127.0.0.1:$leaderPort/kv/delete" -ContentType "application/json" -Body '{"key":"name"}'

index  ok term
-----
12 True  77

(base) PS C:\Windows\System32\WindowsPowerShell\v1.0> try { Invoke-RestMethod -Uri "http://127.0.0.1:$leaderPort/kv/get?key=name" } catch { $_.Exception.Response.StatusCode.value__ }
404
```

图 14: PowerShell KV 操作截图：Put、Get、Delete 与删除后 404

```
C:\Users\PC>curl.exe -X POST http://127.0.0.1:8002/kv/put -H "Content-Type: application/json" -d '{"key":"name","value":"raft"}'
{"index":13,"ok":true,"term":77}

C:\Users\PC>curl.exe http://127.0.0.1:8002/kv/get?key=name
{"found":true,"key":"name","value":"raft"}

C:\Users\PC>curl.exe -X POST http://127.0.0.1:8002/kv/delete -H "Content-Type: application/json" -d '{"key":"name"}'
{"index":14,"ok":true,"term":77}

C:\Users\PC>curl.exe http://127.0.0.1:8002/kv/get?key=name
{"found":false,"key":"name"}
```

图 15: CMD KV 操作截图：curl.exe 完成 Put、Get、Delete

6.2.4 日志复制

连续向 Leader 写入多个 key 后，三个节点的提交位置和已应用位置保持一致，说明日志复制和多数派提交生效。

```
node1 role=Follower term=77 leader=2 commit=17 applied=17 log_len=17
node2 role=Leader term=77 leader=2 commit=17 applied=17 log_len=17
node3 role=Follower term=77 leader=2 commit=17 applied=17 log_len=17
```

Listing 6: 日志复制状态摘要

```
(base) PS C:\Windows\System32\WindowsPowerShell\v1.0> Invoke-RestMethod -Method Post -Uri "http://127.0.0.1:$leaderPort/kv/put" -ContentType "application/json" -Body '{"key":"k1","value":"v1"}'

index  ok  term
-----
15 True  77

(base) PS C:\Windows\System32\WindowsPowerShell\v1.0> Invoke-RestMethod -Method Post -Uri "http://127.0.0.1:$leaderPort/kv/put" -ContentType "application/json" -Body '{"key":"k2","value":"v2"}'

index  ok  term
-----
16 True  77

(base) PS C:\Windows\System32\WindowsPowerShell\v1.0> Invoke-RestMethod -Method Post -Uri "http://127.0.0.1:$leaderPort/kv/put" -ContentType "application/json" -Body '{"key":"k3","value":"v3"}'

index  ok  term
-----
17 True  77

(base) PS C:\Windows\System32\WindowsPowerShell\v1.0> Invoke-RestMethod http://127.0.0.1:8001/status | ConvertTo-Json -Compress
{"id":1,"role":"Follower","term":77,"leader_id":2,"leader_addr":"127.0.0.1:8002","commit_index":17,"last_applied":17,"log_len":17,"last_log_term":77}
(base) PS C:\Windows\System32\WindowsPowerShell\v1.0> Invoke-RestMethod http://127.0.0.1:8002/status | ConvertTo-Json -Compress
{"id":2,"role":"Leader","term":77,"leader_id":2,"leader_addr":"127.0.0.1:8002","commit_index":17,"last_applied":17,"log_len":17,"last_log_term":77}
(base) PS C:\Windows\System32\WindowsPowerShell\v1.0> Invoke-RestMethod http://127.0.0.1:8003/status | ConvertTo-Json -Compress
{"id":3,"role":"Follower","term":77,"leader_id":2,"leader_addr":"127.0.0.1:8002","commit_index":17,"last_applied":17,"log_len":17,"last_log_term":77}
```

图 16: 日志复制截图: 连续写入三个 key 后三节点提交进度一致

```
C:\Users\PC>curl.exe -X POST http://127.0.0.1:8002/kv/put -H "Content-Type: application/json" -d '{"key":"k1","value":"v1"}'
{"index":18,"ok":true,"term":77}

C:\Users\PC>curl.exe -X POST http://127.0.0.1:8002/kv/put -H "Content-Type: application/json" -d '{"key":"k2","value":"v2"}'
{"index":19,"ok":true,"term":77}

C:\Users\PC>curl.exe -X POST http://127.0.0.1:8002/kv/put -H "Content-Type: application/json" -d '{"key":"k3","value":"v3"}'
{"index":20,"ok":true,"term":77}

C:\Users\PC>curl.exe http://127.0.0.1:8001/status
{"id":1,"role":"Follower","term":77,"leader_id":2,"leader_addr":"127.0.0.1:8002","commit_index":20,"last_applied":20,"log_len":20,"last_log_term":77}

C:\Users\PC>curl.exe http://127.0.0.1:8002/status
{"id":2,"role":"Leader","term":77,"leader_id":2,"leader_addr":"127.0.0.1:8002","commit_index":20,"last_applied":20,"log_len":20,"last_log_term":77}

C:\Users\PC>curl.exe http://127.0.0.1:8003/status
{"id":3,"role":"Follower","term":77,"leader_id":2,"leader_addr":"127.0.0.1:8002","commit_index":20,"last_applied":20,"log_len":20,"last_log_term":77}
```

图 17: CMD 日志复制截图: curl.exe 写入三个 key 后三节点 commitIndex 推进到 20

6.2.5 Follower 宕机

保持 node2 为 Leader, 停止一个 Follower 后, node1 与 node2 仍能形成多数派; 继续向 Leader 写入 after_follower_down=ok 可以成功提交并读取。

```
(base) PS C:\Windows\System32\WindowsPowerShell\v1.0> foreach ($p in 8001,8002,8003) {
>> try {
>> Write-Host "==== $p ===="
>> Invoke-RestMethod "http://127.0.0.1:$p/status" | ConvertTo-Json -Compress
>> } catch {
>> Write-Host "==== $p DOWN ===="
>> }
>> }
==== 8001 ====
{"id":1,"role":"Follower","term":80,"leader_id":2,"leader_addr":"127.0.0.1:8002","commit_index":23,"last_applied":23,"log_len":23,"last_log_term":80}
==== 8002 ====
{"id":2,"role":"Leader","term":80,"leader_id":2,"leader_addr":"127.0.0.1:8002","commit_index":23,"last_applied":23,"log_len":23,"last_log_term":80}
==== 8003 DOWN ====
(base) PS C:\Windows\System32\WindowsPowerShell\v1.0> Invoke-RestMethod -Method Post -Uri "http://127.0.0.1:$leaderPort/kv/put" -ContentType "application/json" -Body '{"key":"after_follower_down","value":"ok"}'
index ok term
----
24 True 80

(base) PS C:\Windows\System32\WindowsPowerShell\v1.0> Invoke-RestMethod -Uri "http://127.0.0.1:$leaderPort/kv/get?key=after_follower_down"
found key value
-----
True after_follower_down ok
```

图 18: Follower 宕机截图: node3 无响应时 node2 仍可提交写入

```
(base) PS C:\Windows\System32\WindowsPowerShell\v1.0> Invoke-RestMethod -Method Post -Uri "http://127.0.0.1:$leaderPort/kv/put" -ContentType "application/json" -Body '{"key":"after_follower_down","value":"ok"}'
index ok term
----
21 True 80

(base) PS C:\Windows\System32\WindowsPowerShell\v1.0> Invoke-RestMethod -Uri "http://127.0.0.1:$leaderPort/kv/get?key=after_follower_down"
found key value
-----
True after_follower_down ok
```

图 19: PowerShell Follower 宕机写入截图: 多数派存活时 Put/Get 仍成功

```
C:\Users\PC>curl.exe -X POST http://127.0.0.1:8002/kv/put -H "Content-Type: application/json" -d '{"key":"after_follower_down","value":"ok"}'
{"index":25,"ok":true,"term":80}

C:\Users\PC>curl.exe http://127.0.0.1:8002/kv/get?key=after_follower_down
{"found":true,"key":"after_follower_down","value":"ok"}
```

图 20: CMD Follower 宕机写入截图: curl.exe 写入 after_follower_down 成功

6.2.6 Leader 宕机与新 Leader 可用性

停止当前 Leader node2 后, 8002 端口无响应, 剩余节点重新选举出 node3 作为新 Leader。随后向 node3 写入 after_leader_down=ok 并读取成功, 说明 Leader 故障恢复后系统仍可服务。

```
port 8002 no response
node1 role=Follower term=907 leader=3 commit=27 log_len=27
node3 role=Leader term=907 leader=3 commit=27 log_len=27
put after_leader_down=ok -> ok=True index=28 term=907
```

Listing 7: node2 到 node3 的 Leader 切换摘要

```
(base) PS C:\Windows\System32\WindowsPowerShell\v1.0> foreach ($p in 8001,8002,8003) {
>> try { Invoke-RestMethod -Uri "http://127.0.0.1:$p/status" -TimeoutSec 1 | ConvertTo-Json -Compress }
>> catch { "port $p no response" }
>> }
{"id":1,"role":"Follower","term":907,"leader_id":3,"leader_addr":"127.0.0.1:8003","commit_index":27,"last_applied":27,"log_len":27,"last_log_term":790}
port 8002 no response
{"id":3,"role":"Leader","term":907,"leader_id":3,"leader_addr":"127.0.0.1:8003","commit_index":27,"last_applied":27,"log_len":27,"last_log_term":790}
```

图 21: Leader 宕机截图: node2 无响应, node3 被选为新 Leader

```
C:\Users\PC>curl.exe http://127.0.0.1:8001/status
{"id":1,"role":"Follower","term":907,"leader_id":3,"leader_addr":"127.0.0.1:8003","commit_index":27,"last_applied":27,"log_len":27,"last_log_term":790}
C:\Users\PC>curl.exe http://127.0.0.1:8002/status
curl: (7) Failed to connect to 127.0.0.1 port 8002 after 2052 ms: Could not connect to server
C:\Users\PC>curl.exe http://127.0.0.1:8003/status
{"id":3,"role":"Leader","term":907,"leader_id":3,"leader_addr":"127.0.0.1:8003","commit_index":27,"last_applied":27,"log_len":27,"last_log_term":790}
```

图 22: CMD Leader 宕机截图: node3 接任为 Leader

```
(base) PS C:\Windows\System32\WindowsPowerShell\v1.0> $leader = @(foreach ($p in 8001,8002,8003) {
>> try {
>> $s = Invoke-RestMethod -Uri "http://127.0.0.1:$p/status" -TimeoutSec 1
>> if ($s.role -eq "Leader") { [pscustomobject]@{ Port = $p; Id = $s.id; Addr = $s.leader_addr } }
>> } catch {}
>> }) | Select-Object -First 1
(base) PS C:\Windows\System32\WindowsPowerShell\v1.0> $leaderPort = $leader.Port
(base) PS C:\Windows\System32\WindowsPowerShell\v1.0> Invoke-RestMethod -Method Post -Uri "http://127.0.0.1:$leaderPort/kv/put" -ContentType "application/json" -Body '{"key":"after_leader_down","value":"ok"}'
index ok term
----
28 True 907

(base) PS C:\Windows\System32\WindowsPowerShell\v1.0> Invoke-RestMethod -Uri "http://127.0.0.1:$leaderPort/kv/get?key=after_leader_down"
found key value
----
True after_leader_down ok
```

图 23: PowerShell 新 Leader 可用性截图: node3 接任后 Put/Get 成功

```
C:\Users\PC>curl.exe -X POST http://127.0.0.1:8003/kv/put -H "Content-Type: application/json" -d '{"key":"after_leader_down","value":"ok"}'
{"index":29,"ok":true,"term":907}
C:\Users\PC>curl.exe http://127.0.0.1:8003/kv/get?key=after_leader_down
{"found":true,"key":"after_leader_down","value":"ok"}
```

图 24: CMD 新 Leader 可用性截图: node3 接任后 Put/Get 成功

6.2.7 旧 Leader 恢复与脑裂避免

重新启动旧 Leader node2 后, node2 退回 Follower, node3 仍保持 Leader; 三个节点最终只有一个 Leader, 证明任期、投票和多数派机制能够避免旧 Leader 恢复后形成双主。

```
node1 role=Follower term=907 leader=3 commit=29 log_len=29
node2 role=Follower term=907 leader=3 commit=29 log_len=29
node3 role=Leader term=907 leader=3 commit=29 log_len=29
leaders_after_recovery=1 leader=node3
```

Listing 8: 旧 Leader 恢复后的单 Leader 摘要

```
(base) PS C:\Windows\System32\WindowsPowerShell\v1.0> Invoke-RestMethod http://127.0.0.1:8001/status | ConvertTo-Json -Compress
{"id":1,"role":"Follower","term":907,"leader_id":3,"leader_addr":"127.0.0.1:8003","commit_index":29,"last_applied":29,"log_len":29,"last_log_term":907}
(base) PS C:\Windows\System32\WindowsPowerShell\v1.0> Invoke-RestMethod http://127.0.0.1:8002/status | ConvertTo-Json -Compress
{"id":2,"role":"Follower","term":907,"leader_id":3,"leader_addr":"127.0.0.1:8003","commit_index":29,"last_applied":29,"log_len":29,"last_log_term":907}
(base) PS C:\Windows\System32\WindowsPowerShell\v1.0> Invoke-RestMethod http://127.0.0.1:8003/status | ConvertTo-Json -Compress
{"id":3,"role":"Leader","term":907,"leader_id":3,"leader_addr":"127.0.0.1:8003","commit_index":29,"last_applied":29,"log_len":29,"last_log_term":907}
```

图 25: 旧 Leader 恢复截图: node2 恢复后退回 Follower, node3 仍为唯一 Leader

```
C:\Users\PC>curl.exe http://127.0.0.1:8001/status
{"id":1,"role":"Follower","term":907,"leader_id":3,"leader_addr":"127.0.0.1:8003","commit_index":29,"last_applied":29,"log_len":29,"last_log_term":907}
C:\Users\PC>curl.exe http://127.0.0.1:8002/status
{"id":2,"role":"Follower","term":907,"leader_id":3,"leader_addr":"127.0.0.1:8003","commit_index":29,"last_applied":29,"log_len":29,"last_log_term":907}
C:\Users\PC>curl.exe http://127.0.0.1:8003/status
{"id":3,"role":"Leader","term":907,"leader_id":3,"leader_addr":"127.0.0.1:8003","commit_index":29,"last_applied":29,"log_len":29,"last_log_term":907}
```

图 26: CMD 脑裂避免验证截图: 三个节点最终只有 node3 一个 Leader

```
E:\Tongji learning materials\大三下\大数据存储技术\大作业\raft_kv_codex_task_package>scripts\stop_cluster.bat
Stopping processes listening on Raft KV ports...
Killing PID 69340 on port 8001
Killing PID 14304 on port 8002
Killing PID 57964 on port 8003
Done.
```

图 27: 停止集群截图: 脚本清理节点进程并释放端口

6.2.8 快照字段截图

快照截图在三节点集群连续写入后触发日志压缩, 并展示 Leader 的 `/status` 返回。截图中的关键状态包括快照下标已经推进、保留日志长度小于最后日志下标, 以及快照任期、集群规模和读路径模式等字段。

```
/status: snapshot_index=37 snapshot_term=916 last_log_index=41 log_len=4
/status: cluster_size=3 read_mode=lease_read leader_id=2
```

Listing 9: 快照字段截图应展示的关键字段

```
(base) PS C:\Windows\System32\WindowsPowerShell\v1.0> foreach ($i in 1..9) {
>> Invoke-RestMethod -Method Post -Uri "http://127.0.0.1:$leaderPort/kv/put" -ContentType "application/json" -Body '{"key":"snap$i","value":"v$i"}'
>> }

index  ok  term
-----
33 True 916
34 True 916
35 True 916
36 True 916
37 True 916
38 True 916
39 True 916
40 True 916
41 True 916

(base) PS C:\Windows\System32\WindowsPowerShell\v1.0> Invoke-RestMethod -Uri "http://127.0.0.1:$leaderPort/status" | ConvertTo-Json -Compress
{"id":2,"role":"Leader","term":916,"leader_id":2,"leader_addr":"127.0.0.1:8002","commit_index":41,"last_applied":41,"log_len":4,"last_log_index":41,"last_log_term":916,"snapshot_index":37,"snapshot_term":916,"cluster_size":3,"member":true,"read_mode":"lease_read"}
```

图 28: 快照字段截图: 三节点基础上的快照压缩状态字段

6.2.9 Lease Read 截图

Lease Read 截图需要先向 Leader 写入 `lease_demo=read-ok`, 再执行 Get。验收重点是返回 `found=true`、`value=read-ok` 和 `read_mode=lease_read`, 说明读请求由 Leader 在多数派租约有效时直接读取本地状态机。


```
POST /kv/put {"key":"lease_demo","value":"read-ok"} -> ok=true
GET /kv/get?key=lease_demo -> found=true value=read-ok read_mode=lease_read
```

Listing 10: Lease Read 截图应展示的关键字段

```
(base) PS C:\Windows\System32\WindowsPowerShell\v1.0> Invoke-RestMethod -Method Post -Uri "http://127.0.0.1:$leaderPort/kv/put" -ContentType "application/json" -Body '{"key":"lease","value":"read"}'

index  ok  term
-----
    32  True  916

(base) PS C:\Windows\System32\WindowsPowerShell\v1.0> Invoke-RestMethod -Uri "http://127.0.0.1:$leaderPort/kv/get?key=lease" | ConvertTo-Json -Compress
{"found":true,"key":"lease","read_mode":"lease_read","value":"read"}
```

图 29: Lease Read 截图: 返回 read_mode=lease_read

6.2.10 动态添加节点截图

动态成员变更分成两张截图展示。第一张截图在三节点集群上向 Leader 提交 /cluster/add, 并查询 /cluster, 证明成员列表已经包含 node4; 第二张截图向扩容后的集群写入新 key, 再查询 node4 的 /status, 证明新节点已经追赶到动态写入之后的提交位置。

```
POST /cluster/add node4 -> ok=true
/cluster peers=[1,2,3,4]
POST /kv/put node4=joined -> ok=true
node4 /status: cluster_size=4 commit_index=43 last_applied=43
```

Listing 11: 动态添加节点截图应展示的关键字段

```
(base) PS C:\Windows\System32\WindowsPowerShell\v1.0> Invoke-RestMethod -Method Post -Uri "http://127.0.0.1:$leaderPort/cluster/add" -ContentType "application/json" -Body '{"id":4,"api_addr":"127.0.0.1:8004","raft_addr":"127.0.0.1:9004","data_dir":"data/node4"}'

index  ok  term
-----
    42  True  916

(base) PS C:\Windows\System32\WindowsPowerShell\v1.0> Invoke-RestMethod -Uri "http://127.0.0.1:$leaderPort/cluster" | ConvertTo-Json -Compress
{"peers":[{"id":1,"api_addr":"127.0.0.1:8001","raft_addr":"127.0.0.1:9001","data_dir":"data/node1"}, {"id":2,"api_addr":"127.0.0.1:8002","raft_addr":"127.0.0.1:9002","data_dir":"data/node2"}, {"id":3,"api_addr":"127.0.0.1:8003","raft_addr":"127.0.0.1:9003","data_dir":"data/node3"}, {"id":4,"api_addr":"127.0.0.1:8004","raft_addr":"127.0.0.1:9004","data_dir":"data/node4"}], "status":{"id":2,"role":"Leader","term":916,"leader_id":2,"leader_addr":"127.0.0.1:8002","commit_index":42,"last_applied":42,"log_len":5,"last_log_index":42,"last_log_term":916,"snapshot_index":37,"snapshot_term":916,"cluster_size":4,"member":true,"read_mode":"lease_read"}}
```

图 30: 动态成员变更截图: 添加 node4 后成员列表包含新节点

```
(base) PS C:\Windows\System32\WindowsPowerShell\v1.0> Invoke-RestMethod -Method Post -Uri "http://127.0.0.1:$leaderPort/kv/put" -ContentType "application/json" -Body '{"key":"node4","value":"joined"}'

index  ok  term
-----
43     True  916

(base) PS C:\Windows\System32\WindowsPowerShell\v1.0> Invoke-RestMethod -Uri "http://127.0.0.1:8004/status" | ConvertTo-Json -Compress
{"id":4,"role":"Follower","term":916,"leader_id":2,"leader_addr":"127.0.0.1:8002","commit_index":43,"last_applied":43,"log_len":6,"last_log_index":43,"last_log_term":916,"snapshot_index":37,"snapshot_term":916,"cluster_size":4,"member":true,"read_mode":"lease_read"}
```

图 31: 动态成员变更截图: node4 启动后追赶到最新提交位置

6.2.11 自动化验收与画图结果

为避免截图只能证明单次现象, 本项目新增自动验收流程。该流程使用临时端口和临时数据目录启动真实节点, 依次验证核心 KV/Raft 功能、快照、Lease Read、动态添加节点和五节点多数派行为。测试结束后会生成结构化记录和 SVG 图表, 作为截图之外的可重复证据。

```
run automated validation
== core raft kv functions ==
== five-node status ==
== snapshot, lease read, dynamic add ==
PASS=true
```

Listing 12: 2026-06-26 自动验收输出摘要

自动化验收结果显示: 核心 Put/Get/Delete、Follower 重定向、Follower 宕机继续写、Leader 宕机重选主均通过; 快照场景中 `snapshot_index=8`、`log_len=3`; Lease Read 返回 `read_mode=lease_read`; 动态添加 node4 后四个节点的 `commit_index` 均追到 13。五节点场景中, 系统启动 5 个真实节点并选出唯一 Leader, 停止两个 Follower 后仍可形成 $Q(5) = 3$ 的多数派并完成 Put/Get。

6.2.12 可视化控制台截图

前端控制台截图展示了同一套 Raft 状态如何从终端命令转化为浏览器中的可观察界面。第一张图连接到 node1 的 API 端口, 但页面能够识别 node1 当前为 Follower、node2 为 Leader, 并在节点表中展示每个成员的任期、提交位置、日志长度和快照下标。图中 node4 处于 Down 状态, 说明控制台不仅展示配置中的成员, 也能直观标识暂未启动或不可达的节点; 右侧响应面板保留最近一次接口返回, 活动面板记录 Put/Get 操作已被路由到 Leader。

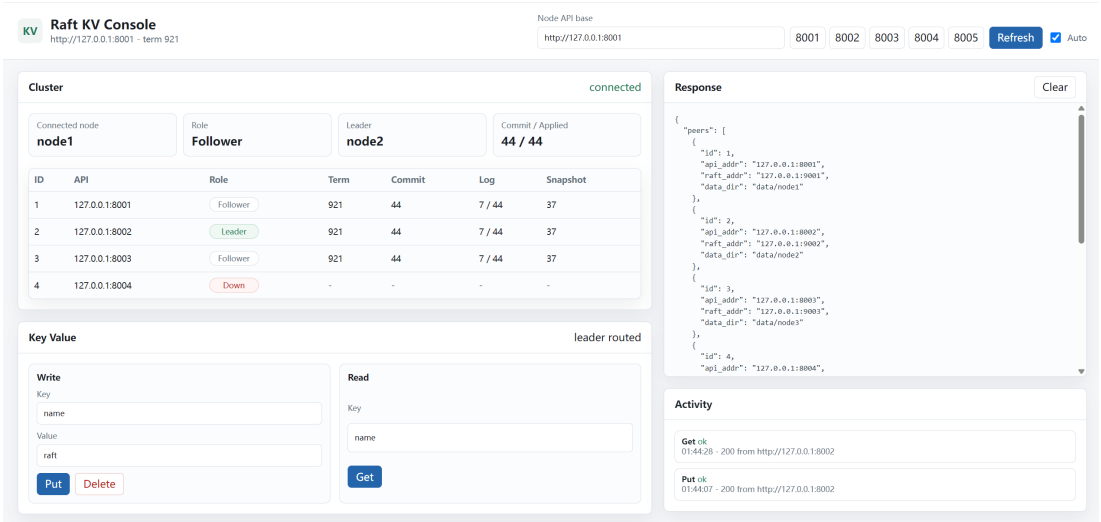


图 32: 可视化控制台截图：集群状态、快照字段、Leader 路由与 KV 操作

第二张图展示成员管理面板。添加节点时需要输入 ID、API 地址、Raft 地址和数据目录；移除节点时只需要指定成员 ID。该界面对应后端已经实现的教学版动态成员变更接口，操作仍由 Leader 通过 Raft 日志提交后生效，而不是在前端直接修改本地配置。

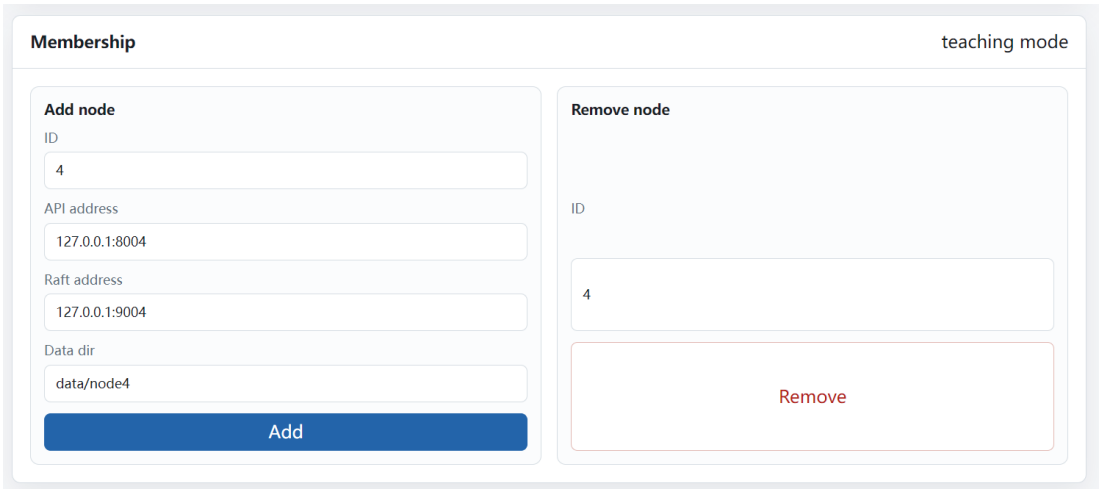


图 33: 可视化控制台截图：动态成员添加与移除面板

6.3 自动化测试结果

在当前工程中，完整测试使用项目内 Go 构建缓存执行，避免 Windows 默认缓存目录权限影响测试结果：

```
$env:GOCACHE = (Resolve-Path .).Path + ".\gocache"  
D:\Go\bin\go.exe test ./...
```

测试全部通过。单元测试验证了 KV 状态机语义、Raft 投票规则、AppendEntries 日志匹配、冲突截断、提交推进、快照安装和成员变更日志应用；集成测试启动真实 HTTP/RPC 多节点服务，覆盖 Follower 重定向、Leader 故障重选主、持久化重启恢复、五节点扩展容错、Lease Read、快照可见性和动态添加节点。新增前端测试进一步验证 /ui/ 可以由运行中的节点直接提供，且 CORS 预检能够支持浏览器跨端口访问当前 Leader。由于服务层测试使用真实端口和真实持久化目录，它比单纯 mock 调用更接近课程演示时的运行状态。

6.4 故障恢复实验结果

三节点实验首先验证选主安全性：启动后集群只产生一个 Leader，其余节点保持 Follower。随后执行 Put、Get、Delete，写入值能够被读取，删除后再次读取返回 404；连续写入多个 key 后，三个节点的 commitIndex 与 lastApplied 同步推进，说明日志复制和状态机应用顺序一致。

在单节点故障场景中，停止一个 Follower 后仍有 2 个节点存活，满足三节点多数派条件 $Q(3) = 2$ ，因此 Leader 可以继续提交写入。相反，当三节点中两个 Follower 同时停止时，存活节点数小于多数派，写请求无法提交并返回 commit timeout。这一结果符合 Raft 的安全边界：系统宁可拒绝提交，也不能在没有多数派确认时产生可能回滚的写入。

Leader 故障场景验证了系统可用性和防脑裂能力。停止旧 Leader 后，剩余节点会进入新一轮选举并产生新的唯一 Leader，新 Leader 可以继续处理 Put/Get；旧 Leader 恢复后会通过更高任期的 RPC 退回 Follower，最终 leaders_after_restart=1。这说明任期单调递增、单任期单投票和多数派选举共同阻止了双 Leader。

6.5 快照、租约读与成员变更验证

持久化恢复实验在写入数据后停止全部节点，再重新启动三节点集群。新 Leader 产生后，原先写入的 key 仍能读取，说明本地持久化日志和快照可以恢复状态机。快照实验中，连续写入触发日志压缩，snapshot_index 推进，快照边界之前的日志被截断，保留日志长度小于最后日志索引。该结果说明系统不会随着写入次数增加而无限保留旧日志。

Lease Read 实验验证了读路径的低开销一致性读取。Leader 在租约有效时直接读取本地状态机并返回 read_mode=lease_read；Follower 收到读写请求时仍返回

Leader 信息，避免客户端从落后副本读取不完整状态。动态成员变更实验通过 Raft 日志提交添加 node4，新节点启动后可以通过 AppendEntries 或 InstallSnapshot 追赶到 Leader 的提交位置。可视化实验进一步证明，浏览器控制台展示的角色、提交位置、快照字段和成员状态来自真实 HTTP 接口，KV 写入和成员变更仍按 Leader 路由与 Raft 日志提交路径执行。

7 局限性与未来方向

当前系统面向课程实验和可演示性设计，已经覆盖 Raft 主流程、故障恢复、快照、Lease Read、动态成员变更和前端可视化，但与生产级分布式存储系统相比仍存在边界。首先，成员变更采用教学版实现，能够展示“配置变化必须通过 Raft 日志提交后生效”的核心思想，但尚未实现 joint consensus，因此不适合在复杂并发扩缩容场景中直接使用。其次，Lease Read 依赖本地时钟租约假设，报告和测试验证了读路径行为，但还没有给出严格时钟漂移约束下的完整证明；若要面向生产环境，应进一步实现标准 ReadIndex 或补充更严格的租约安全条件。

性能方面，当前日志复制以清晰性为优先，未实现批量 AppendEntries、连接复用、异步 apply 和冲突索引快速回退。在写入压力较高或网络延迟较大的环境中，这些机制会直接影响吞吐和恢复速度。运维方面，系统已经提供状态接口和 Web 控制台，但仍缺少权限控制、审计日志、指标采集、告警和系统化网络分区模拟。未来可以围绕安全成员变更、严格一致读、复制性能优化、可观测性和故障注入测试展开，使系统从课程演示原型进一步接近工程实践。

8 总结

本项目完成了课程作业要求的核心功能：可配置多节点集群、Leader 选举、心跳、RequestVote、AppendEntries、日志复制、多数派提交、KV 状态机、持久化恢复和故障切换。测试结果表明，系统可以在 Follower 宕机和 Leader 宕机情况下继续提供服务，旧 Leader 恢复后也不会形成双 Leader。核心安全性来自任期单调递增、单任期单投票、日志匹配检查和多数派提交条件 $Q(N) = \lfloor N/2 \rfloor + 1$ 。

项目同时实现了快照压缩、Lease Read、教学版动态成员变更、真实 HTTP 多节点集成测试以及内置 Web 控制台。实验截图、自动化测试和可视化页面共同表明，系统不仅能够在命令行中完成核心协议验证，也能够以更直观的方式展示节点角色、提交进度、快照字段、Leader 路由和成员变化。实现重点放在 Raft 主流程的完整性、可运行性、可视化观察和可重复验证上。

9 参考文献

1. Diego Ongaro and John Ousterhout. *In Search of an Understandable Consensus Algorithm*. USENIX Annual Technical Conference, 2014. <https://www.usenix.org/conference/atc14/technical-sessions/presentation/ongaro>
2. Diego Ongaro. *Consensus: Bridging Theory and Practice*. PhD dissertation, Stanford University, 2014. <https://web.stanford.edu/~ouster/cgi-bin/papers/OngaroPhD.pdf>
3. The Go Authors. *Package net/http*. Go Documentation. <https://pkg.go.dev/net/http>
4. The Go Authors. *Package encoding/json*. Go Documentation. <https://pkg.go.dev/encoding/json>
5. CTEX. *ctex – LaTeX classes and packages for Chinese typesetting*. CTAN. <https://ctan.org/pkg/ctex>